

Preparcial: liquidaciones de tarjetas de crédito

Una entidad financiera recibe un CSV de consumos de una tarjeta de crédito correspondientes a un año y mes determinados. El lote es de una sola tarjeta: el archivo no mezcla marcas. El programa carga movimientos válidos y calcula la comisión que se liquida por cada consumo. Este proyecto se entrega para estudiar y ejecutar; no hay tareas de implementación para esta instancia.

Ejecución

Requiere JDK 21 o 25 y Maven 3.9.x. Desde esta carpeta:

```
mvn test
java -cp target/classes utnfc.backend.parcial.Main
java -cp target/classes utnfc.backend.parcial.Main datos/datos.csv
```

La primera ejecución de Maven puede requerir Internet. Una vez resueltas las dependencias, probar `mvn -o test` en la misma computadora. El programa no requiere dependencias externas en ejecución.

Contrato del archivo

UTF-8, encabezado exacto `id,rubro,importeCentavos,bonificacionCentavos,estado`. Cada registro ocupa una línea con cinco campos separados por coma. Los campos no admiten comas, comillas ni saltos de línea internos; no hay campos entrecomillados. Se quitan espacios externos de cada campo. Es un formato de intercambio controlado, no un lector de todo CSV posible. El uso de `split(",", -1)` conserva los campos finales vacíos y es válido bajo este contrato.

Cada fila tiene identificador y rubro no vacíos, importe y bonificación enteros en centavos, y estado. No se exige unicidad de identificadores; cada fila válida cuenta como un consumo independiente. Los rubros son textos sensibles a mayúsculas (por ejemplo `Alimentos`, `Combustible`, `Servicios`).

El importe debe estar entre 1 y 5_000_000 centavos (\$1 a \$50.000); la bonificación debe ser no negativa y menor al importe. El neto es importe menos bonificación y debe estar entre 1 y 3_000_000 centavos (\$1 a \$30.000). `Consumo.desdeCampos` convierte los campos textuales a enteros y llama al constructor. El constructor recibe importe y bonificación, calcula el neto y valida esas reglas. El parser clasifica filas (ancho, estado, continuidad) y no concentra la interpretación de montos.

La comisión es el **2 % del neto**, en centavos. Se calcula con enteros: $(\text{neto} \times 2) / 100$. **La fracción de centavo se desprecia**: no se redondea, no se cobra el centavo siguiente y no se usa punto flotante. Ejemplos: 49 → 0; 50 → 1; 100_000 → 2000; 100_001 → 2000 (los 0,02 centavos sobrantes no se cobran); 3_000_000 → 60000.

Flujo y errores

El parser verifica primero la cantidad de columnas. Con ancho correcto, `ANULADO` se descarta sin validar los restantes datos. `APROBADO` se intenta convertir; cualquier otro estado es inválido. Una línea vacía es inválida. No se interrumpe la carga por errores de una fila: se registra el número físico de línea como clave y el motivo como valor, en mapas separados para anulados e inválidos. Un encabezado incorrecto aborta con `IllegalArgumentException`; un fallo de lectura propaga `IOException`. El encabezado no cuenta como fila leída.

`procesadas` significa filas aceptadas, no intentos de conversión. Se cumple `leídas = procesadas + descartadas + inválidas` y `objetos = procesadas`. Se conserva el orden de los objetos aceptados y de

los diagnósticos.

Recorrido de lectura sugerido

1. Ejecutar `Main` y los tres grupos de tests.
2. Leer `Consumo`: constructor, `desdeCampos`, invariantes y comisión derivada.
3. Seguir `ParserLiquidaciones` y `ResultadoParseo`: clasificación y manejo de excepciones.
4. Leer `LiquidadorTarjetas`: copia defensiva, `Predicate<Consumo>`, filtros, total, promedio, máximos/mínimos y agrupamiento por rubro.
5. Localizar casos testigo en el archivo y explicar su clasificación y comisión; usar el programa para verificar los agregados del conjunto.

El archivo contiene 60 filas de datos más encabezado (61 líneas). Panorama esperado: 60 leídas, 39 procesadas, 6 descartadas y 15 inválidas; total 636156; rubros Alimentos 210000, Combustible 188081, Servicios 238075; ocho consumos de hasta \$1000. Casos testigo: C01 (100_000 centavos netos, comisión 2000), C06 (3_000_000, comisión 60000), C07 (anulado con monto cero), C09 (neto cero inválido), I013 (1 centavo, comisión 0) e I049 (3_000_100, inválido).

Los getters se escriben explícitamente: no se agrega Lombok para minimizar dependencias y configuración del entorno de examen. `ResultadoParseo` también usa getters convencionales y conserva copias defensivas de la lista de consumos y de los mapas de diagnósticos. `JUnit 5` es la única dependencia y se usa solamente en tests.